SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

backend - JSP

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. JSP 소개 및 특징
2. MVC 디자인 패턴
3. MVC 실습

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

JSP

Confidential

❖ 기존 Servlet의 문제점

```
protected void gugu(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
 int dan = Integer.parseInt(request.getParameter("dan"));
  StringBuilder html = new StringBuilder("""
    <html>
      <body>
        <tab1e>
        <tbody>
      """);
  for (int i = 1; i < 10; i++) {
    html.append("<tr><td>%d * %d</td><td>%d</td></tr>".formatted(dan, i, dan * i));
  }
  html.append("""
        </tbody>
      <table>
    </body>
  </html>""");

  response.getWriter().append(html);
}
```

문자열로 작성 하다 보니
자동완성도 안되고..
디자이너님!! 뭔가 이상해요. ㅠㅠ



3 * 133 * 263 * 393 * 4123 * 5153 * 6183 * 7213 * 8243 * 927

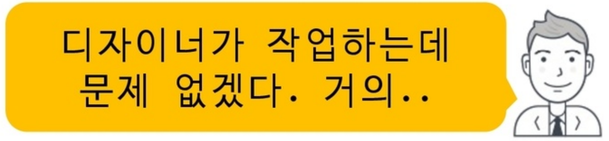
Confidential

❖ JSP

- Jakarta Server Pages
  - ◆ Servlet이 Java 코드 사이에 HTML 태그를 작성했다면 JSP는 HTML 태그 기반에 Java 코드를 작성하는 형태

```
<%
  try {
    int dan = Integer.parseInt(request.getParameter("dan"));
%>
  <table>
    <tbody>
<% for (int i = 1; i < 10; i++) { %>
      <tr>
        <td><%=dan + " * " + i%></td>
        <td><%=dan * i%></td>
      </tr>
<%}%>
    </tbody>
  </table>
<%
} catch (NumberFormatException e) {
  e.printStackTrace();
  out.println("<h1>dan 파라미터를 입력하세요.</h1>");
}
%>
```

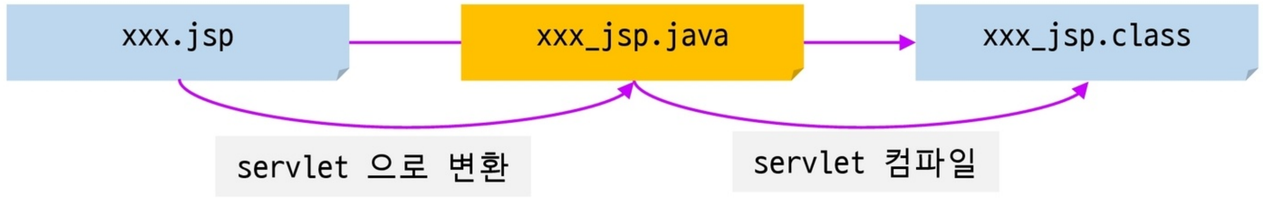
business logic
+ presentation logic



Confidential

❖ JSP의 동작 원리

- Servlet과의 관계: JSP는 중간단계! 최종은 Servlet

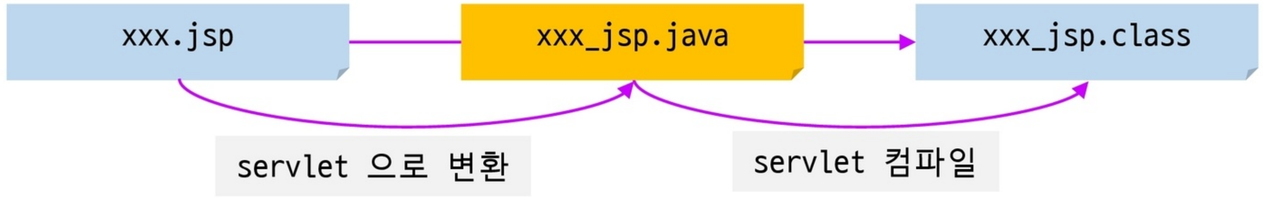


- 생성된 자바 파일의 위치
  - ◆ [workspace]/metadata/plugins/org.eclipse.wst.server.core/tmp0/work/Catalina/localhost
  - ◆ 생성된 자바 파일에서 확인할 내용들
    - Servlet Life Cycle에 해당하는 메서드
    - service 메서드의 local 변수들

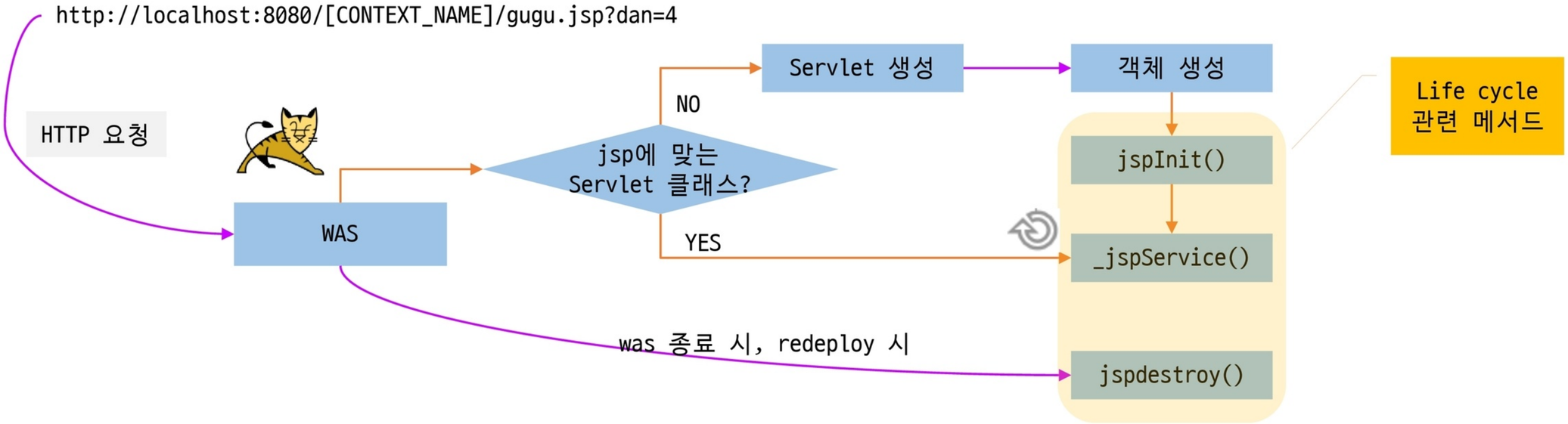
Confidential

❖ JSP의 동작 원리

- Servlet과의 관계: JSP는 중간단계! 최종은 Servlet



- JSP Life Cycle



Confidential

❖ JSP 구성 요소

요소	설명	표현
directive	JSP 페이지의 속성 지정	<%@ . . . %>
script-let	자바 실행문 기술 →_jspService 내부로 치환됨	<% . . . %>
expression	계산식, 함수의 호출 결과를 출력할 때	<%= . . . %>
declaration	변수 및 메서드 선언 →class 영역 구성	<%! . . . %>
comment	JSP 페이지 내의 주석	<%-- . --%>
built-in object	기본적으로 제공되는 로컬 변수들	request, …
expression language	디자이너도 하는 쉬운 스크립트	${expr}
JSTL	custom 태그 중 자주 사용되는 것을 표준화 (JSP Standard Tag Library)	

Confidential

❖ directive - JSP 지시자

- JSP 페이지가 실행될 때 필요한 정보를 지정하는 역할
  - ◆ 필요한 정보를 컨테이너에게 알려서 서블릿 생성에 활용
- 형태
  - ◆ <%@ [directive] 속성="값" 속성="값"…%>의 형태

- 주요 directive

directive	설명	용도
page	JSP 페이지에 대한 기본 정보 지정 문서의 타입, 출력 버퍼의 크기, 에러 페이지 설정 등	모든 JSP 필수
taglib	JSP 페이지에서 사용할 태그 라이브러리 지정	JSTL
include	JSP 페이지의 특정 영역에 다른 문서 포함	page 모듈화 및 재사용

Confidential

❖ page directive

- JSP page에 대한 기본 정보 지정

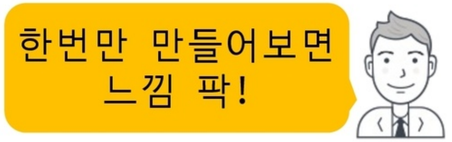
속성	기본값	설명	사용 예
pageEncoding	UTF-8	페이지의 문자 인코딩 설정	<%@ page pageEncoding="UTF-8" %>
contentType	text/html;charset=ISO8859-1	페이지의 MIME 타입 설정	**<%@ page contentType="text/html;charset=UTF-8" %>**
import		**페이지에서 사용할 클래스의 패키지**	**<%@ page import="java.util.*" %>**
language	java	페이지에서 사용하는 언어	<%@ page language="java" %>
session	true	세션 사용 여부 설정	<%@ page session="true" %>
buffer	8kb	버퍼 크기 설정	<%@ page buffer="16kb" %>
autoFlush	true	버퍼 자동 플러시 여부 설정	<%@ page autoFlush="false" %>
errorPage		오류 발생 시 이동할 페이지 설정	<%@ page errorPage="error.jsp" %>
isErrorPage	false	오류 페이지 여부 설정	<%@ page isErrorPage="true" %>

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="java.util.*, java.io.*" %>
```

JSP 작성

Confidential

❖ JSP 스크립트 요소 - Java 코드 작성



- script-let: <% [자바 실행문] %>
  - ◆ _jspService 메서드 내부에 삽입됨 → local 영역
  - ◆ local 영역에 선언된 JSP 내장 객체(HttpServletRequest, HttpServletResponse, JspWriter, …)를 자유롭게 사용

- declaration: <%! [멤버 변수 또는 메서드] %>
  - ◆ 선언 위치에 상관 없이 멤버 변수, 메서드 선언

- expression: <%= [출력 할 내용] %>
  - ◆ <%=변수%>, <%=리턴이 있는 메서드 호출%>, <%=수식%>
  - ◆ Servlet에서 _jspService() 내부에서 out.print() 형태로 변형

- comment: <%-- [주석 내용] --%>
  - ◆ JSP에서만 존재하며 Servlet에는 전달되지 않음
  - ◆ html 주석을 이용해서 민감한 내용이 화면에 전달되지 않도록 주의!!

JSP 작성

Confidential

❖ today.jsp

```jsp
<%@ page import="java.time.LocalDateTime"%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <body>
  <%-- script-let: 일반 자바 코드 작성 --%>
  <%
    LocalDateTime now = LocalDateTime.now();
    out.println("현재 시간 : " + getFormatted(now));
  %>

  <%-- expression: 출력이 필요할 때 --%>
  <%=getFormatted(now)%>

  <%-- declaration: member 선언 --%>
  <%!private String getFormatted(LocalDateTime now) {
    return String.format("%tY-%<tm-%<td (%<ta)", now);
  }%>
  <!-- 기존의 HTML 주석 활용 -->

</body>
</html>
```

이 jsp가 servlet에서는 어떻게 표현될까?



Confidential

❖ Built-in Object(내장 객체)?

- .jsp 파일의 _jspService 메서드 내부에 로컬 변수로 이미 선언 된 객체
  - ◆ 스크립트릿 영역에서 동일한 이름으로 로컬 변수를 선언할 수 없음

변수명	변수 타입	용도
response	jakarta.servlet.http.HttpServletResponse	서버의 응답 정보
out	jakarta.servlet.jsp.JspWriter	서버의 응답 정보 출력 스트림
pageContext	jakarta.servlet.jsp.PageContext	현재 JSP 페이지에 대한 정보
request	jakarta.servlet.http.HttpServletRequest	클라이언트 요청 정보
session	jakarta.servlet.http.HttpSession	HTTP 세션에 대한 정보
application	jakarta.servlet.ServletContext	웹 애플리케이션에 대한 정보
exception	java.lang.Throwable	isErrorPage=true인 경우 사용되는 예외 객체
config	jakarta.servlet.ServletConfig	JSP 페이지에 대한 설정 정보 저장

Confidential

❖ 상태 관리와 Web scope

- standalone application의 경우 local 변수, member 변수, class 변수를 통해 정보 관리
- 네트워크 너머에 있는 server에서 애플리케이션이 동작하는 상황에서는 위와 같은 scope를 사용하기 어려움

page - 1회용 컵



한 번 쓰고 버리는 1회용 컵
(한 페이지 내에서만 유효)

request - 주문



주문에서 음식 서빙
완료까지만 유지

session - 자리 번호

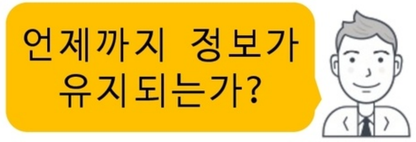


내가 식당을 나갈 때까지
(로그아웃까지) 유지

application - 식당 규칙



모든 손님이 공유하는 규칙

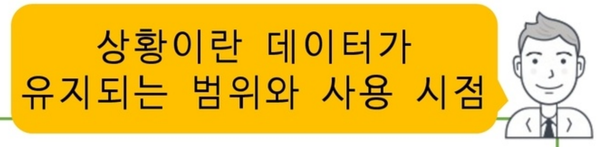


제작: Gemini

Confidential

❖ Web scope - 추가적인 속성(attribute) 저장을 위한 웹의 영역

- 상황에 따라 데이터를 저장하고 공유하며 생명주기를 관리하는데 사용



접근 영역	설명			
page scope	특징	JSP 페이지 내에서 유효한 속성 저장. 같은 페이지 내에서는 접근 가능, 페이지가 종료되면 소멸		
	Servlet	없음	JSP	pageContext 내장 객체
request scope	특징	요청(Request) 처리 동안 유효한 속성 저장. 응답이 완료되면(Response) 해당 속성은 소멸		
	Servlet	doGet, doPost의 파라미터인 HttpServletRequest request	JSP	request 내장 객체
session scope	특징	사용자가 웹 애플리케이션에 접속한 동안 유효한 속성 저장, 세션이 종료/만료되면 해당 속성 소멸		
	Servlet	HttpSession session = request.getSession();	JSP	session 내장 객체
application scope	특징	웹 애플리케이션 전체에 걸쳐 유효한 속성 저장, 모든 사용자의 세션에서 공유		
	Servlet	ServletContext application = getServletContext();	JSP	application 내장 객체

Confidential

❖ Web scope - 사용 방법

메서드	설명
void setAttribute(String key, Object value)	속성을 지정한 키와 값으로 저장 - 기존 속성 덮어쓰기 가능
Object getAttribute(String key)	지정한 키에 해당하는 속성의 값을 반환 - 값이 존재하지 않으면 null 반환
void removeAttribute(String key)	지정한 키에 해당하는 속성을 제거
Enumeration<String> getAttributeNames()	모든 속성의 키를 Enumeration으로 반환

```jsp
<%-- attribute 설정 --%>
<%
  request.setAttribute("reqAttr", "reqAttr");
  session.setAttribute("sesAttr", "sesAttr");
  application.setAttribute("appAttr", "appAttr");
%>
```

```jsp
<h2>attribute 확인</h2>
<ul>
  <li>request: <%=request.getAttribute("reqAttr") %></li>
  <li>session: <%=session.getAttribute("sesAttr") %></li>
  <li>application: <%=application.getAttribute("appAttr") %></li>
</ul>
```

Request Parameter vs Attribute

Confidential

❖ Request Parameter vs Attribute

	Parameter	Attribute
출처	클라이언트에서 전송 폼, URL, AJAX 등	서버에서 저장 request.setAttribute()로 설정
설정 방법	GET: /main?name=kim POST: form data	request.setAttribute("key", value)
조회 방법	request.getParameter("name") getParameter("key") → String	request.getAttribute("key") getAttribute() → Object
데이터 수정	수정 불가(읽기 전용)	수정 가능 remove(), setAttribute() 가능
사용 시기	사용자 입력값 폼 데이터, 검색어	처리한 데이터 DB 조회 결과, 중간 값

Confidential

❖ Servlet VS JSP

구분	서블릿 (Servlet)	JSP (JavaServer Pages)
실행 과정	컴파일된 .class 파일을 웹 컨테이너가 직접 실행	최초 요청 시 서블릿 코드로 변환 변환된 서블릿 코드를 컴파일 컴파일된 서블릿을 실행 (이후 요청 시 이 단계만 반복)
핵심 구성	Java 코드 내에 HTML이 포함된 형태	HTML 코드 내에 Java 코드가 포함된 형태
개발 생산성	UI(화면) 작업이 매우 복잡하고 번거로움.	화면 구성이 직관적이고 쉬움. HTML처럼 작성하다 필요한 부분에만 자바 코드 삽입



누구를 써야 할까?

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

MVC 디자인 패턴

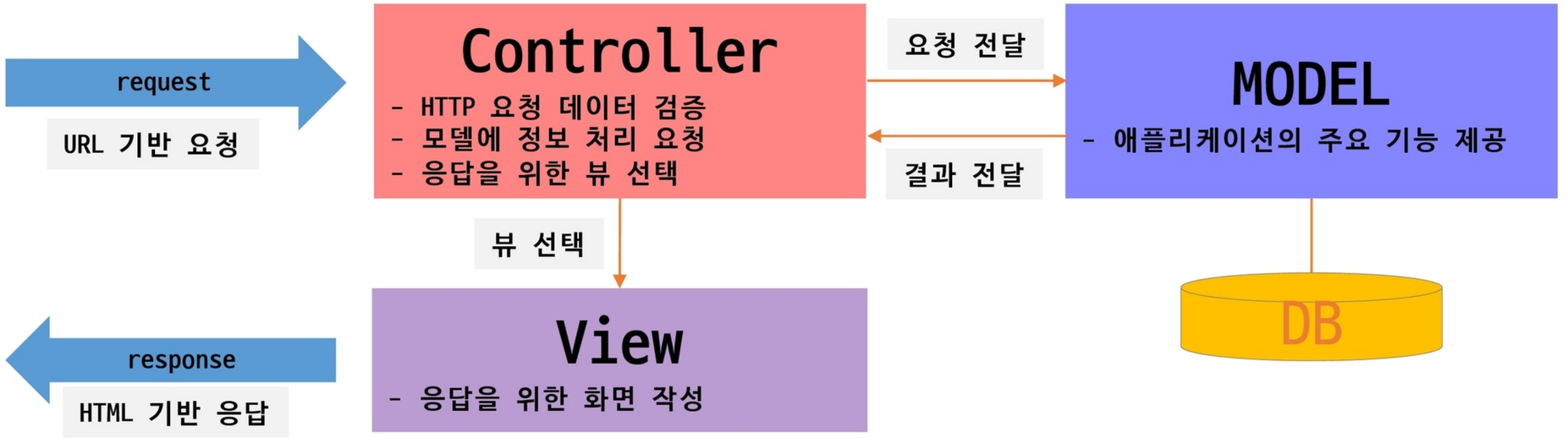
Confidential

❖ MVC 디자인 패턴

- 애플리케이션을 역할에 따라 Model, View, Controller로 모듈화 하는 패턴
  - ◆ Controller: 사용자의 요청을 받고 Model과 View 간의 상호 작용을 조정하는 모듈
  - ◆ Model: 비즈니스 로직 즉 업무(Service)를 담당하는 모듈로 필요 시 DB 연동(DAO)
  - ◆ View: 사용자와의 상호작용을 위한 UI를 담당하는 모듈

Java Web에서 MVC 패턴은
Model 1과 Model 2 방식이 있다.





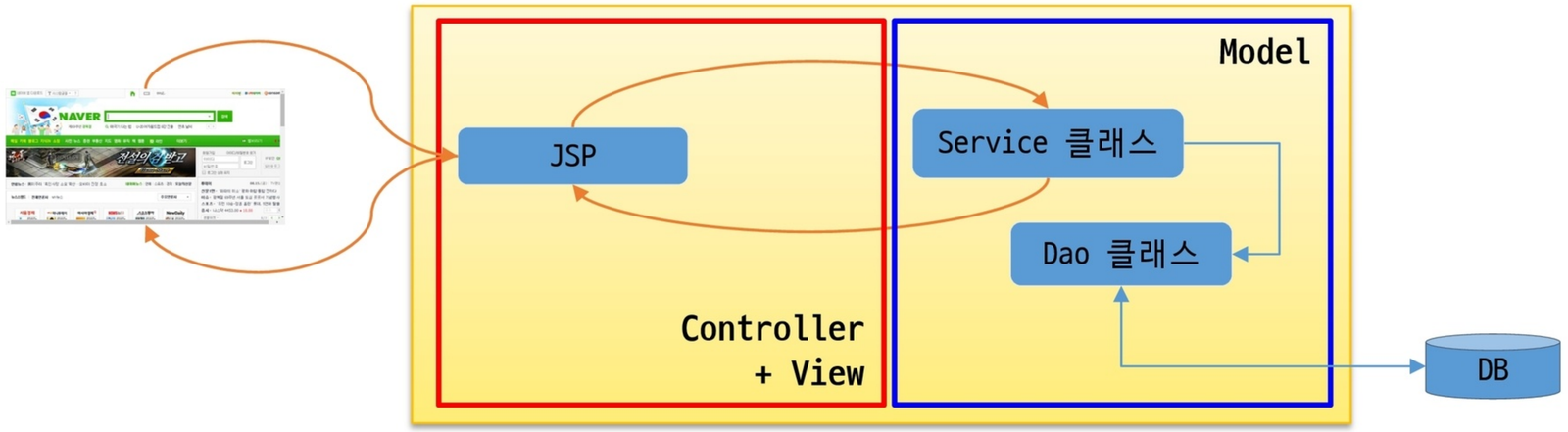
- 장점
  - ◆ 분리된 관심사로 독립적 개발 및 유지보수가 용이
  - ◆ 코드의 가독성, 재사용성 향상

MVC 디자인 패턴

Confidential

❖ Model 1 방식

- JSP가 Controller를 맡아 View 로직과 함께 수행



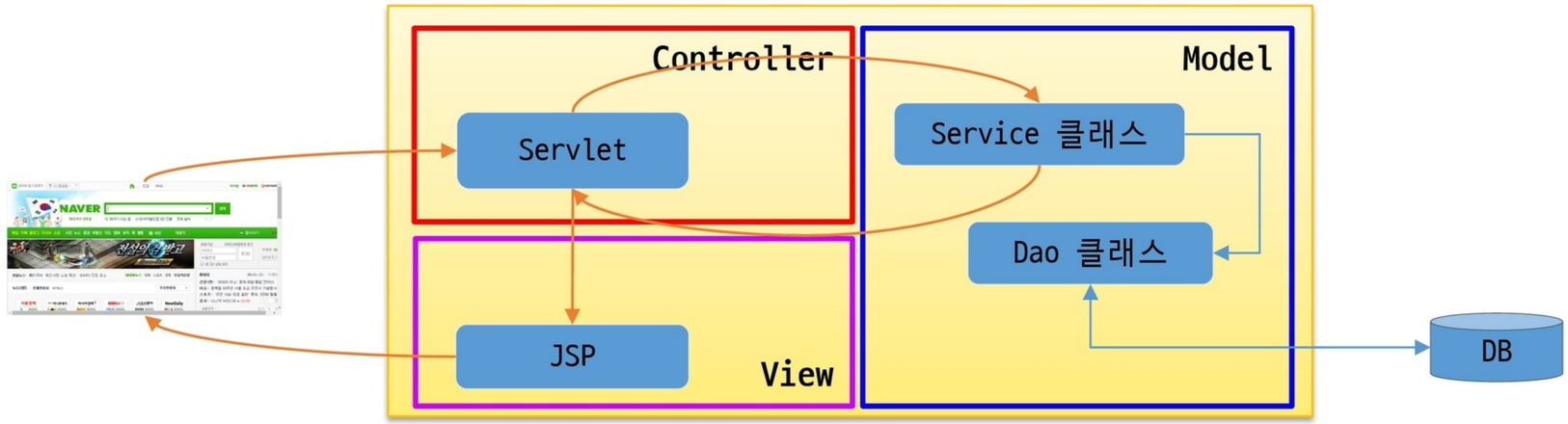
장점	단점
• 단순성: 구조가 간단하여 소규모 애플리케이션에 적합 • 빠른 개발: 초기 개발 속도가 빠름 • 직관적: JSP 하나만 사용하므로 학습 곡선이 완만함	• 유지보수 어려움: 비즈니스 로직과 프레젠테이션 로직이 혼합되어 있어 코드가 복잡해지고 유지보수가 어려우며 재사용성 저하 • 확장성 제한: 애플리케이션이 커질수록 구조가 복잡해지고 확장성이 떨어짐

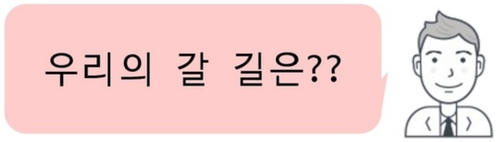
MVC 디자인 패턴

Confidential

❖ Model 2

- 엄격히 MVC 패턴을 따르며 Servlet이 Controller를 맡아 클라이언트 요청 접수, 전체적인 조절, JSP는 화면만 관리





장점	단점
• 명확한 구조: 각 구성 요소가 명확히 분리되어 있어 코드의 가독성 향상 • 유지보수 용이: 비즈니스 로직과 프레젠테이션 로직이 분리되어 있어 유지보수가 용이하며 코드 재사용성 향상 • 확장성: 규모가 커져도 구조가 명확하여 확장하기 용이	• 복잡성: 구조가 복잡해져 초기 설정과 개발이 다소 어려움 • 개발 속도: Model1 방식에 비해 초기 개발 속도가 느림 • 학습 곡선: MVC 패턴에 대한 이해가 필요하므로, 개발자에게 학습 곡선이 가파름

MVC 디자인 패턴

Confidential

❖ Model 2 Deep Dive

이제 모든 요청은 Servlet이 받는다!



Controller: Servlet

주요 업무

- HttpServletRequest 분석
- 파라미터를 이용해 적절한 Service 호출
- Model의 반환 값 확인
- 결과를 적절한 웹 스코프에 attribute로 저장
- 상황에 따라 적절한 View 호출

Model: Java

Service

주요 업무
- 비즈니스 로직 처리
- 필요시 DAO 호출로 DB 연동

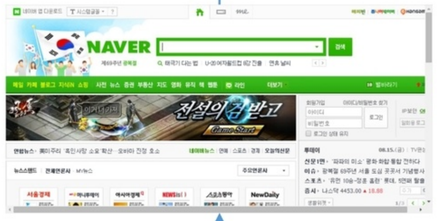
DAO

주요 업무
- JDBC를 이용해 DB 연동

jdbc

DB

근처 맛집!



View: JSP

C는 어떻게 V를 호출하지?



주요 업무
- 웹 스코프에 저장된 attribute를 활용해 화면 생성
- 클라이언트에 화면 전달

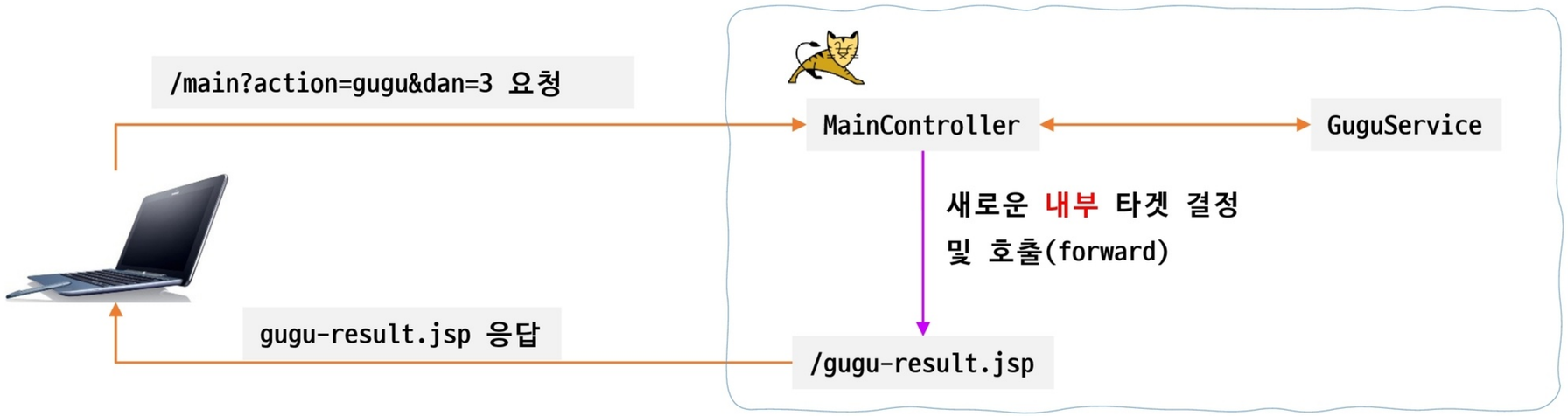
웹 컴포넌트 호출

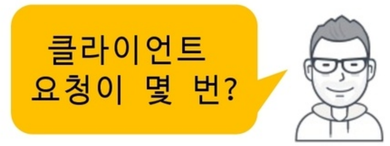
Confidential

❖ 웹 컴포넌트 호출: forward

- 서블릿, JSP는 메서드 호출처럼 직접 호출하지 않음: container가 호출
- 호출 요청을 위해 RequestDispatcher#forward나 HttpServletResponse#sendRedirect 사용

- RequestDispatcher#forward
  - ◆ controller가 받았던 request와 response를 JSP에 전달 - request의 정보는 JSP에서 확인 가능





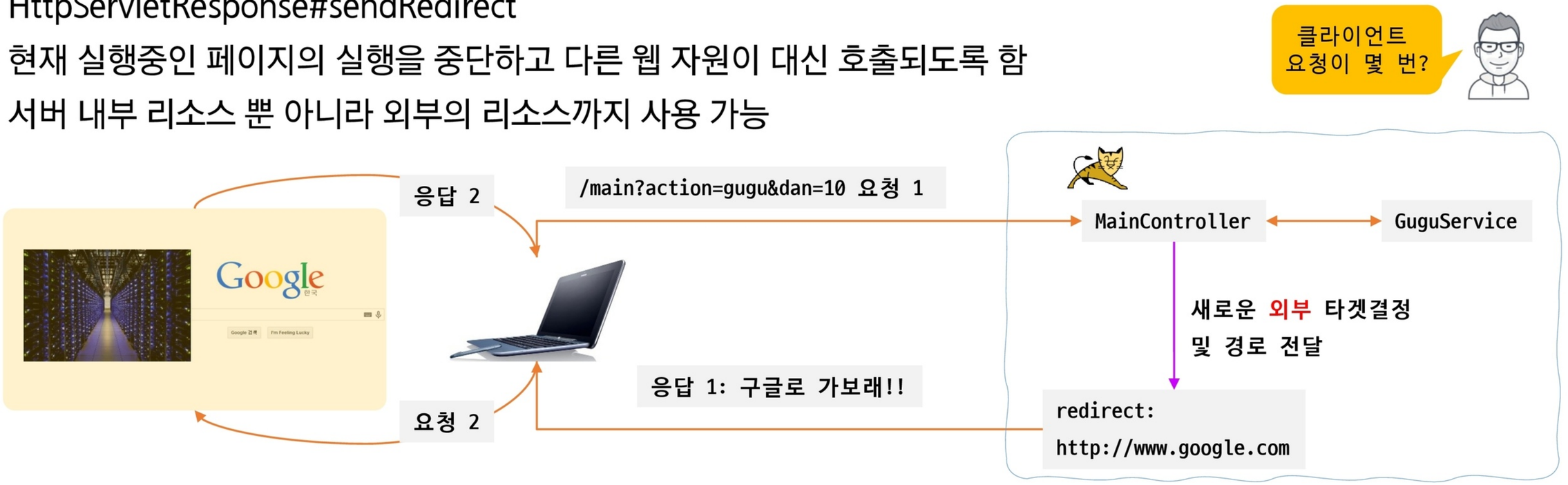

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/gugu-result.jsp");
dispatcher.forward(request, response);
```

웹 컴포넌트 호출

Confidential

❖ 웹 컴포넌트 호출: redirect

- HttpServletResponse#sendRedirect
- 현재 실행중인 페이지의 실행을 중단하고 다른 웹 자원이 대신 호출되도록 함
- 서버 내부 리소스 뿐 아니라 외부의 리소스까지 사용 가능




```
response.sendRedirect("http://www.google.com");
```

- 클라이언트가 최종적으로 받은 응답 2는 요청 1과는 무관
  - ◆ 요청 1에 저장한 정보는 요청2를 전달받은 곳에서 확인할 수 없음

웹 컴포넌트 호출

Confidential

❖ sendRedirect vs forward

response.sendRedirect()	dispatcher.forward()
• URL을 웹 브라우저로 보내서 간접 호출 • 최초 요청과 무관한 request 와 response를 새로 생성	• 웹 서버에서 직접 호출 • 최초 발생한 request와 response 전달
• 클라이언트 입장에서는 2번의 호출 - 클라이언트 브라우저의 URL이 변경됨	• 클라이언트 입장에서는 1번의 호출 - 클라이언트 브라우저의 URL은 처음 호출한 URL 유지
• 다른 웹 애플리케이션의 자원 호출 가능	• 동일 웹 애플리케이션의 자원만 호출
• 2차 요청 시 parameter로 문자열만 전달 가능 - queryString으로 redirect url에 추가했을 경우 • 필요 시 session attribute 사용 → session 관리 부담	• request parameter로 문자열 전달 가능 • request attribute로 객체 전달 가능
• /는 container root	• /는 context root

ControllerHelper 수정

Confidential

❖ 페이지 이동을 위한 utility method 추가(TODO 01, 02)

index.jsp를 forward와 redirect로 호출해보자.



```
public interface ControllerHelper {

  public default String preProcessing(HttpServletRequest req, HttpServletResponse resp) {
    . . .
    return action;
  }

  public default void redirect(HttpServletRequest request, HttpServletResponse response, String path)
                                                                                    throws IOException {
    if (path.startsWith("http")) {
      response.sendRedirect(path);
    } else {
      response.sendRedirect(request.getContextPath() + path);
    }
  }

  public default void forward(HttpServletRequest request, HttpServletResponse response, String path)
                                                                          throws ServletException, IOException {
    RequestDispatcher disp = request.getRequestDispatcher(path);
    disp.forward(request, response);
  }
}
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

MVC 실습

MVC 작성하기

Confidential

❖ 일반적인 작업 순서

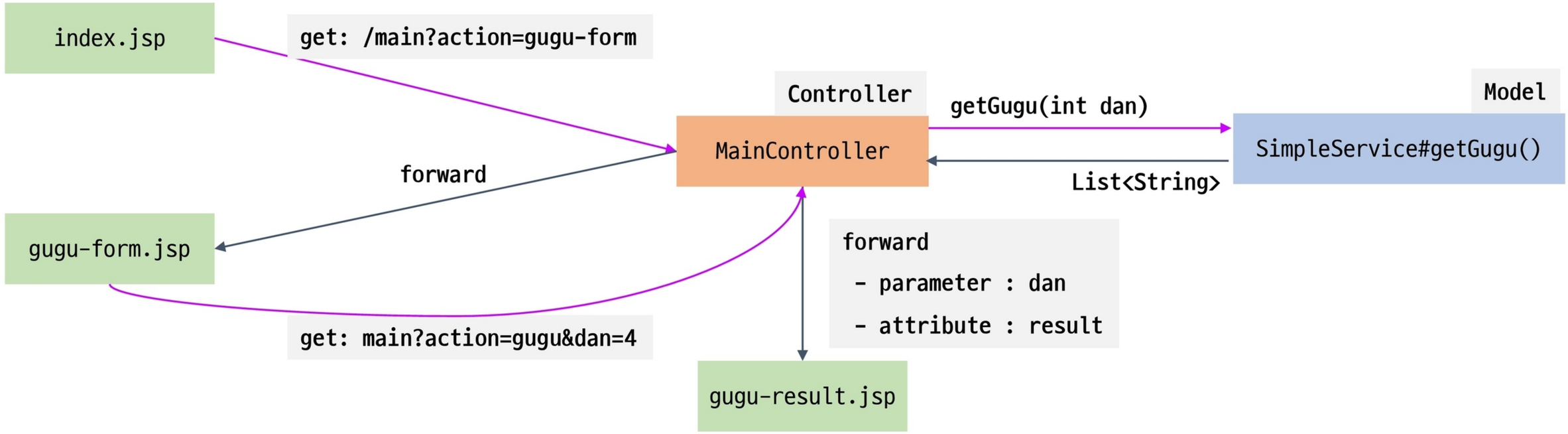
- Model 작성: 비즈니스 로직 구성 - 화면에서 보여줄 데이터 결정
- View 작성: 데이터 활용
- Controller 작성: 접착제처럼 두 부분을 이어주기

- 유즈케이스 별 작성
  - ◆ 회원 가입 Model -> View -> Controller
  - ◆ 회원 목록 Model -> View -> Controller
  - ◆ . . .

다시 보는 gugudan(TODO: 03 ~ 10)

Confidential

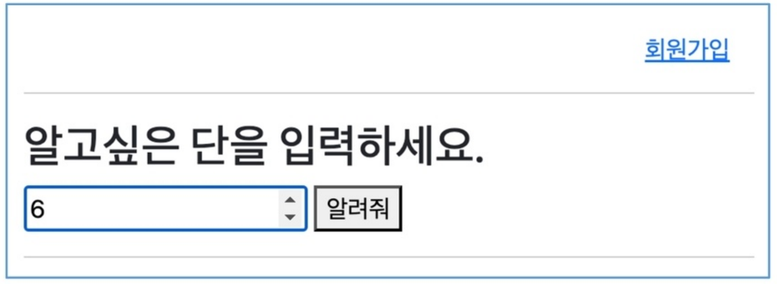
❖ MVC 버전의 gugudan 흐름 : 성공



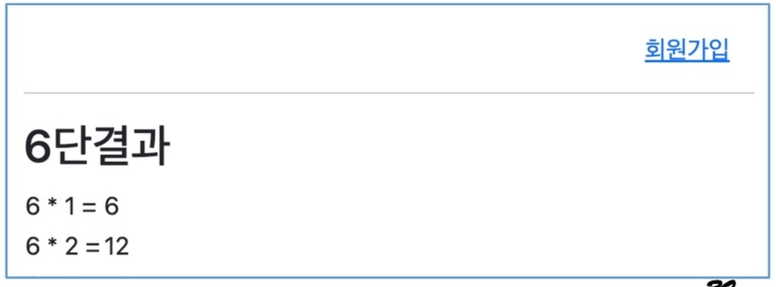
index.jsp



gugu-form.jsp



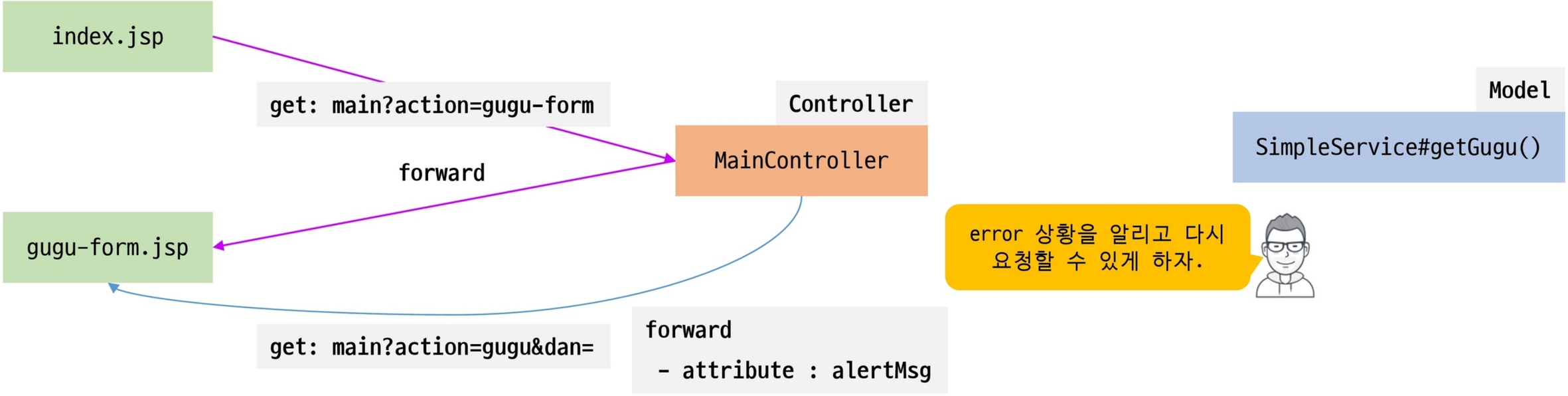
gugu-result.jsp

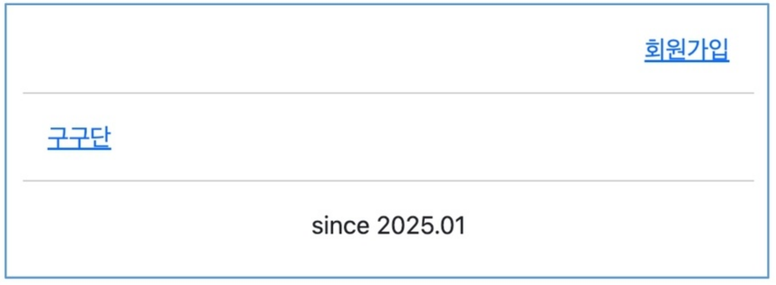


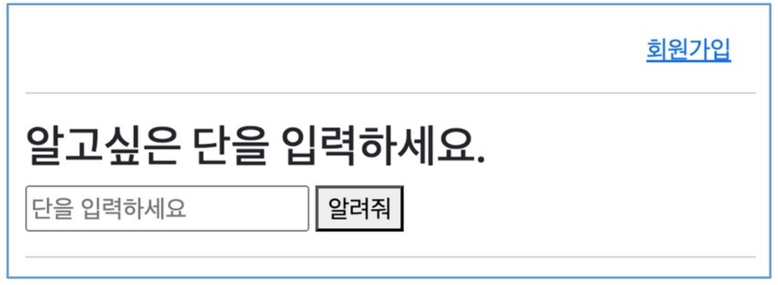
다시 보는 gugudan(TODO:11, 12)

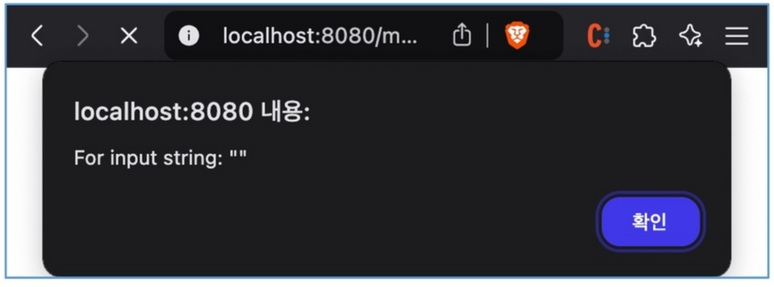
Confidential

❖ MVC 버전의 gugudan 흐름 : 입력 오류에 의한 실패





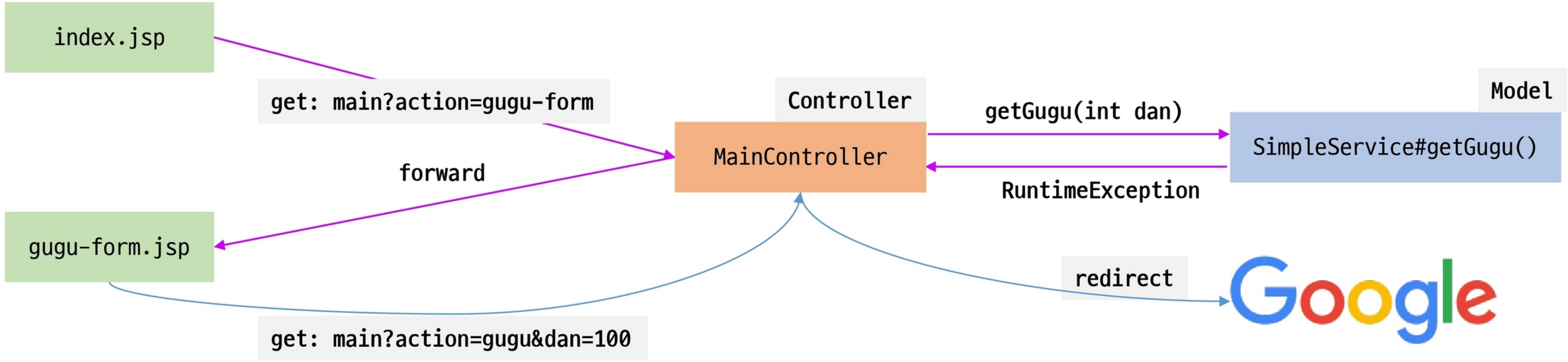




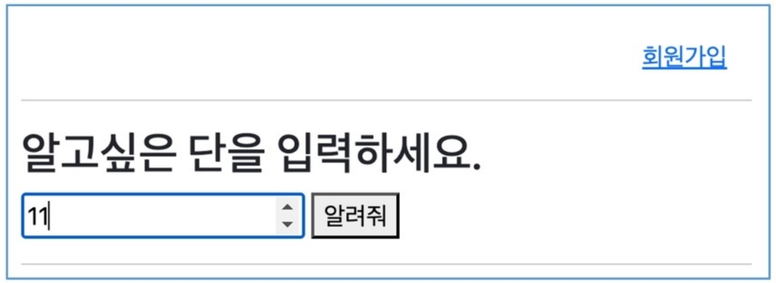
다시 보는 gugudan(TODO:13)

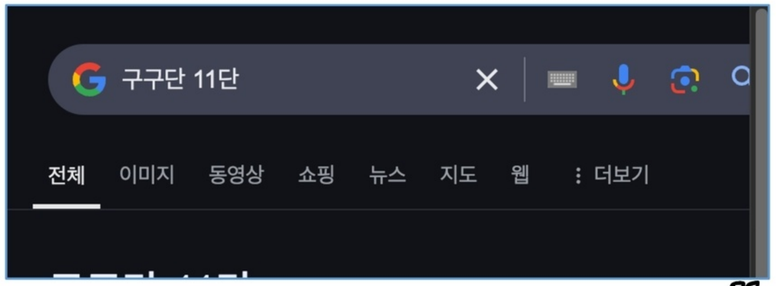
Confidential

❖ MVC 버전의 gugudan 흐름 : 구구단의 범위를 넘어가면 구글에 물어보기









내일 수업을 위해서..

Confidential

❖ 실습 파일 설치

- 수업 git에서 BE_04_lab.zip 가져오기
- 사용할 workspace에 압축 풀기(여기에 압축 풀기)



- STS : [File] - [Open Projects from File System]
  - ◆ index.jsp 동작 확인

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미